

Introduction to Digital Computer Systems II

Akingbade, Michael Adeniyi

Obafemi Awolowo University, Ile - Ife Nigeria

akingbadem@oauife.edu.ng

May 9, 2024

Overview

- 1 Logic Gates
 - NAND Gate
 - NOR Gate
 - Exclusive OR Gate
 - Exclusive NOR Gate
- 2 Computational Logic
- 3 Sequential Logic
- 4 Flip Flop
 - NAND LATCH
 - S-R Flip-Flop
 - NOR GATE LATCH
- 5 Counters
- 6 Registers
- 7 PAL and PAL

Computational and Sequential Logic

Computational Logic

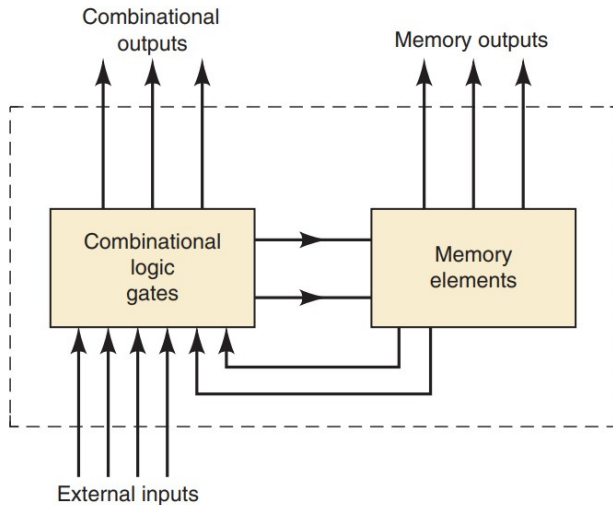
- The output of a combinational logic depends only on current input.
- It has no memory.

Sequential Logic

- In sequential logic, the present output depends not only on current input but also on past input values.
- It needs some type of memory to remember the past input values.

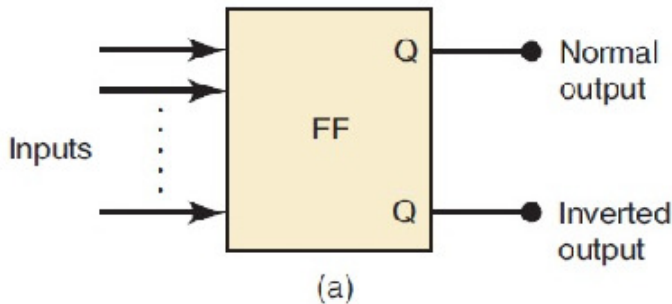
General Digital System.

- Most digital systems consists of both combinational circuits and memory elements.



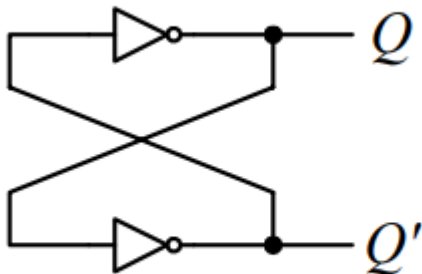
- Flip-Flop is a fundamental building block of fundamental building block of sequential logic circuits..
- it is a bistable multivibrator.
- it has two stable states and
- It can remain in either state indefinitely until it is triggered to change
- It is used for storing binary information and for clocked synchronous circuit design.

General Flip Flop Symbol.



The image above is the Flip Flop symbol

Cross Coupled Inverter.



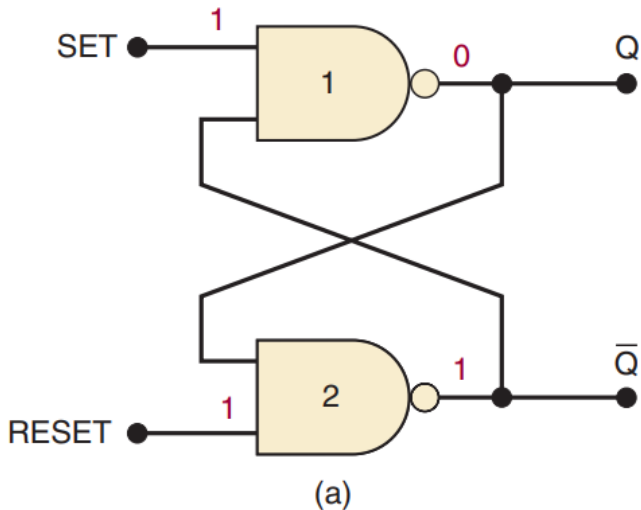
The image above is the Cross Coupled Inverter

NAND GATE LATCH

S-R LATCH

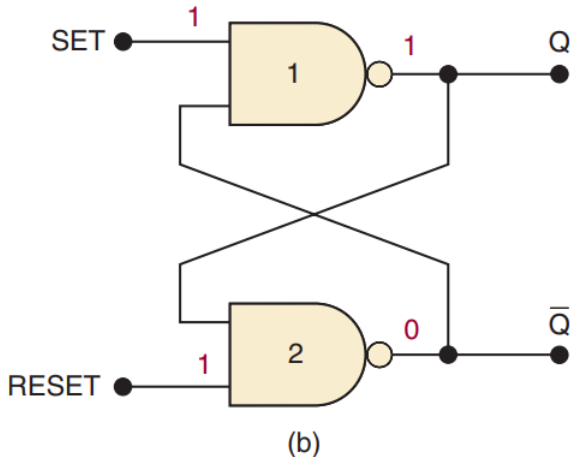
- The two NAND gates are cross coupled so that the output, of NAND-1 is connected to one of the inputs of NAND-2.
- and vice-versa
- The Latch outputs are labeled Q and $\bar{Q}$.
- There are two latch inputs: e Latch inputs:.
- The **SET input** is the input that sets Q to the 1 state.s:.
- The **RESET input** is the input that resets Q to the 0 state.

NAND Latch.



A NAND latch has two possible resting states when $\text{SET} = \text{RESET} = 1$.
In the first case, Q will produce 0 state

NAND Latch.



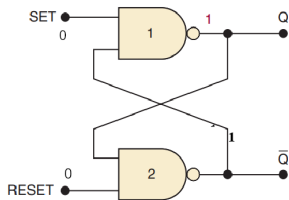
A NAND latch has two possible resting states when $\text{SET} = \text{RESET} = 1$.
In the second case, Q will produce 1 state

When NAND Latch is ($\text{SET} = \text{RESET} = 0$, Unpredictable Results)

When the **SET** and **RESET** inputs return HIGH, the resulting output state will depend on which inputs returns High first.

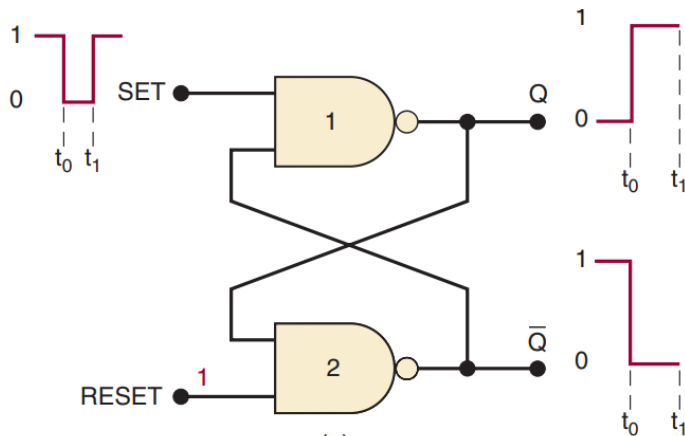
Simultaneous transitions back to the 1 state will produce **unpredictable results**.

For these reasons the **SET** = **RESET** = 0 condition is normally not used for the NAND latch.



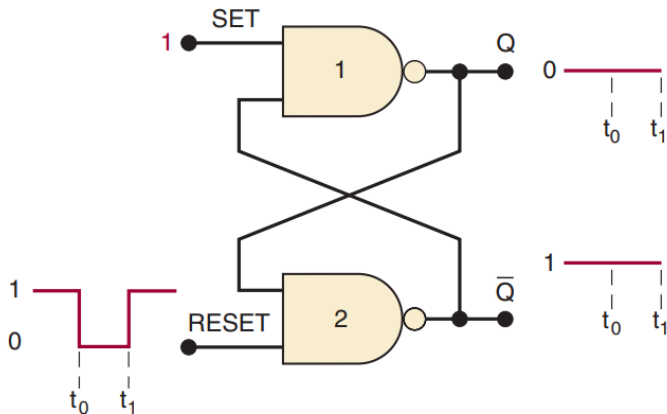
A NAND latch has unpredictable results when $\text{SET} = \text{RESET} = 0$

Setting the Latch (When $SET = 0$ and $RESET = 1$)



Stating that a low pulse on the SET input will always cause the Latch to end up in the $Q=1$ state.

Resetting the Latch (When $SET = 1$ and $RESET = 0$)



Resetting the Latch(Clearing); when $S=1$; $R=0$

Stating that a low pulse on the RESET input will always cause the Latch to end up in the $Q=0$ state.

NAND SR LATCH Truth Table.

Function Table

SET	RESET	OUTPUT
1	1	No change
0	1	$Q=1$
1	0	$Q=0$
0	0	Invalid

NAND LATCH Summary

SET = RESET = 1

(1) SET = RESET = 1:

- This condition is the only normal resting state, and
- it has no effect on the output state.
- The Q and $\bar{Q}$ output will remain in whatever state they were in prior to this input condition.

SET = 0, RESET=1

(2) SET = 0, RESET=1:

- This will always cause the output Q=1 state,
- where it will remain even after SET returns HIGH.
- This is called setting the Latch.

NAND LATCH Summary

SET = 1, RESET = 0

(3) SET = 1, RESET=0:

- This will always produce the $Q=0$ state;
- where the output will remain even after RESET returns High.
- This is called the clearing or resetting the Latch.

SET = RESET = 0

(4) SET = RESET=0:

- This condition tries to set and clear the Latch at the same time
- and it produces $Q = \bar{Q} = 1$.
- if the inputs are returned to 1 simultaneously,.
- the resulting state is unpredictable. This input condition should not be used.

Introduction to Digital Computer Systems II

Akingbade, Michael Adeniyi

Obafemi Awolowo University, Ile - Ife Nigeria

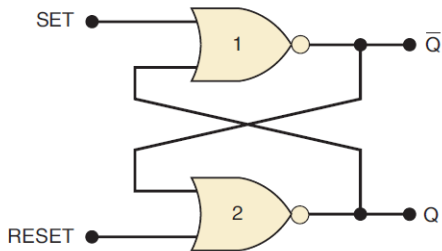
akingbadem@oauife.edu.ng

June 5, 2024

Overview

- 1 Flip Flop
 - S-R Flip-Flop
 - NOR GATE LATCH
- 2 Counters
- 3 Registers
- 4 PAL and PAL

NOR GATE Latch



NOR GATE TRUTH TABLE

Set	Reset	Output
0	0	No change
1	0	$Q = 1$
0	1	$Q = 0$
1	1	Invalid*

*Produces $Q = \overline{Q} = 0$.

(b)

NOR GATE Latch

SET = RESET = 0

- (1) SET = RESET = 0:
- This is the normal resting state for the latch and it has no effect on the output state.
- Q and $\bar{Q}$ will remain in whatever state they were in prior to the occurrence of this condition.

SET = 1, RESET = 0

(2) SET = 1, RESET = 0:

- This will always set Q = 1, where it remains even after SET returns to 0.

NOR GATE Latch

SET=0, RESET=1

(3) SET=0, RESET=1:

- This will always clear $Q=1$, where it remains even after RESET returns to 0.

SET=1, RESET=1

(4) SET=1, RESET=1:

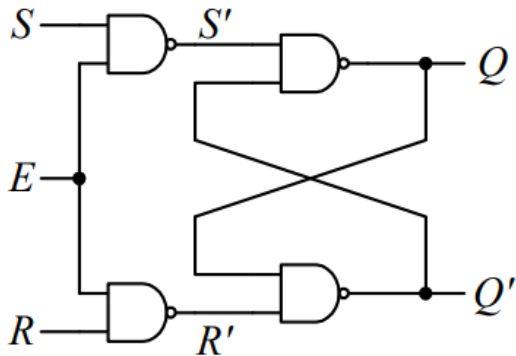
- This condition tries to set and reset the latch at the same time and it produces $Q = \bar{Q} = 0$.
- If the inputs are returned to 0 simultaneously, the resulting output state is unpredictable.

Function Table for NOR GATE Latch

S	R	Q	$\bar{Q}$	Comments
0	0	Q	$\bar{Q}$	Hold Condition
1	0	1	0	Flip Flop set
0	1	0	1	Flip Flop reset
1	1	0	0	Not used

Gated S-R Latch

- Simple gate circuits, combinational logic, transparent S-R Flip Flops are called asynchronous because the output responds immediately to changes.
- Synchronous circuit operate sequentially in step, with a control input.
- To make an S-R flip flop synchronous, we add a gate input to enable and disable the S and R inputs.
- The flip flop will operate normally while the gate enable is High



The image above is the Gated S-R Flip Flop.

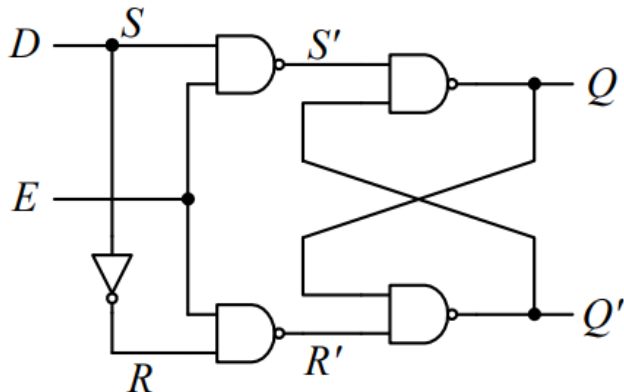
The flip flop will operate normally while the gate enable is High. E is used to enable or disable the latching effect. It is only when E is 1 that 1 from S OR R GET THROUGH TO THE CROSS CONNECTED S-R only

Function Table for Gated S-R Flip flop

G	S	R	Q	$\bar{Q}$	Comments
0	0	0	Q	$\bar{Q}$	Hold (S - R Inputs disabled)
0	0	1	Q	$\bar{Q}$	Hold (S - R Inputs disabled)
0	1	0	Q	$\bar{Q}$	Hold (S - R Inputs disabled)
0	1	1	Q	$\bar{Q}$	Hold (S - R Inputs disabled)
1	0	0	Q	$\bar{Q}$	Hold (S - R Inputs enabled)
1	0	1	0	1	Reset (S - R Inputs enabled)
1	1	0	1	0	Set (S - R Inputs enabled)
1	1	1	0	0	Unused (S - R Inputs enabled)

Gated D Latch

The gated D latch is obtained by simply inverting the input of S (in the SR latch) with an inverted.



Truth Table of D Flip Flop

Clk	D	Q_{n+1}
0	0	0
1	0	0
1	1	1

The characteristic equation of D flip flop

$$Q_{next} = D$$

Characteristic Table for the D Flip Flop

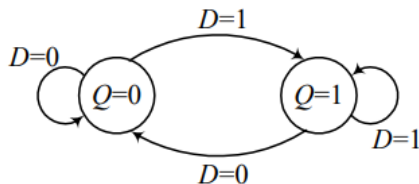
Characteristic Table for the D Flip Flop

Q_n	D	Q_{n+1}
0	0	0
0	1	1
1	0	0
1	1	1

D Flip Flop Characteristic Table

D	Q_n	Q_{n+1}
0	x	0
1	x1	1

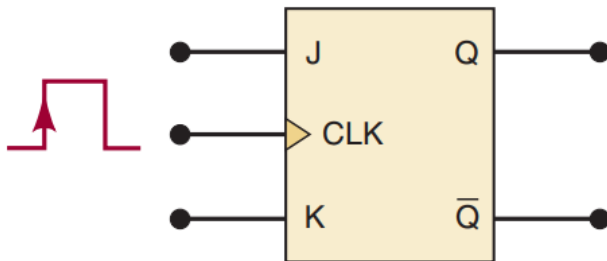
State Diagram and Excitation Table of the D Flip Flop



State diagram of D flip-flop

Excitation Table for the D Flip Flop

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1



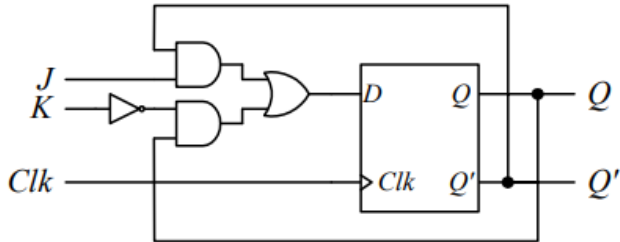
Logic Symbol of J-K flip-flop that responds only to the positive edge of the clock

If $J = K = 1$

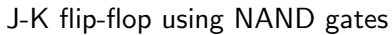
The condition does not result in an ambiguous output.

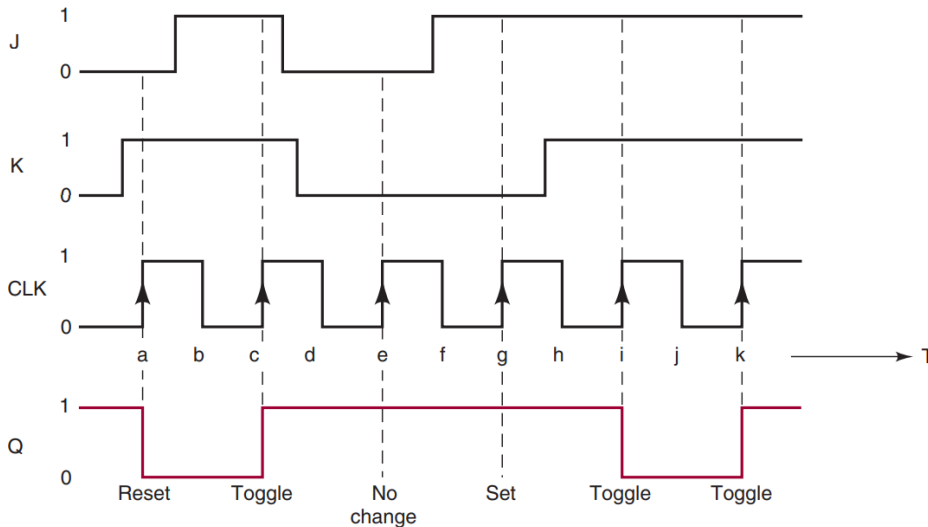
When $J = K$ are both 1, the flip flop will always go to its opposite state upon the positive transition of the clock signal.

This is called the toggle mode of operation.



Circuit of J-K flip-flop





Clocked J-K flip-flop waveforms

The operation of the flip flop

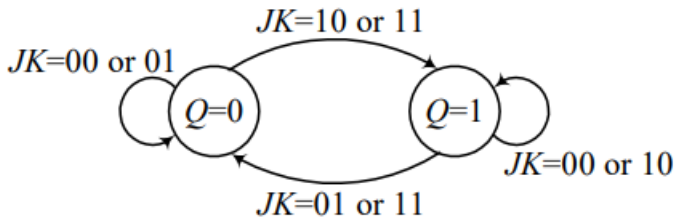
- Initially all inputs are 0, and the Q output is assumed to be 1; that is $Q_0 = 1$.
- When the positive going edge of the first clock pulse occurs (point a), the $J = 0, K = 1$ condition exists. The flip flop will reset to the $Q = 0$ state.
- The second clock pulse finds $J = K = 1$. When it makes its positive transition (point c). This causes the flip flop to toggle to its opposite state, $Q = 1$.
- At point e on the clock waveform, J and K are both 0, so that the flip flop does not change states on this transition.
- At point g, $J = 1$ and $K = 0$. This is the condition that sets Q to the 1 state. However, it is already in 1, and so it will remain there.
- At point i, $J = K = 1$, and so the flip flop toggles to its opposite state. The same thing occurs at point k.

Characteristic Table for the J -K Flip Flop

Q_n	J	K	Q_{n+1}	$\bar{Q}_{n+1}$
0	0	0	0	1
0	0	1	0	1
0	1	0	1	0
0	1	1	1	0
1	0	0	1	0
1	0	1	0	1
1	1	0	1	0
1	1	1	0	1

The characteristic equation of JK flip flop.

$$Q_{next} = J\bar{Q} + Q\bar{K}$$



State diagram of JK flip flop

J	K	CLK		Q
0	0	↑		Q_0 (no change)
1	0	↑		1
0	1	↑		0
1	1	↑		$\overline{Q_0}$ (toggles)

Clocked J-K flip-flop Characteristic table

Truth Table for the J-K Flip Flop

J	K	Q_{n+1}	
0	0	Q_n	(memory)
0	1	0	1
1	0	1	0
1	1	$\bar{Q}_n$	

The operation of the JK flip-flop is very similar to the SR flip-flop. The J input is just like the S input in the SR flip-flop in that, when asserted, it sets the flip-flop.

Similarly, the K input is like the R input where it resets the flip flop when asserted. The only difference is when both inputs, J and K, are asserted.

For the SR flip-flop, the next state is undefined; whereas, for the JK flip-flop, the next state is the inverse of the current state. In other words, the JK flip-flop toggles its state when both inputs are asserted.

Excitation Table for the J -K Flip Flop

Q_n	Q_{n+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

SR FLIP FLOP

SR Flip flop change their content only at the active edge of clock signal
SR flip flops can enter an undefined state when both inputs are asserted simultaneously. When the two inputs are de-asserted, then the next state is the same as the current state.

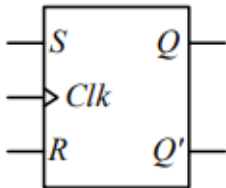
Characteristic Table of the SR Flip Flop

S	R	Q_n	Q_{n+1}	$\overline{Q_{n+1}}$
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	x	x
1	1	1	x	x

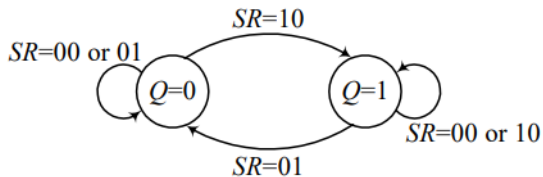
Q_{next}		SR			
		00	01	11	10
Q	0	0	0	x	1
	1	1	0	x	1

The characteristic equation is the boolean equation that is derived directly from the characteristic table.

$$Q_{next} = S + \bar{R}Q$$



S-R Logic Symbol



S-R state diagram

Q	Q_{next}	S	R
0	0	0	$\times$
0	1	1	0
1	0	0	1
1	1	$\times$	0

S-R excitation table

T FLIP FLOP

T FLIP FLOP

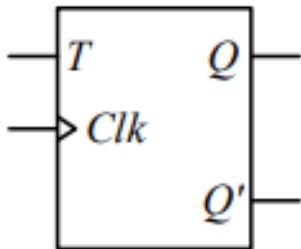
The T Flip flop has one input, T (toggle), in addition to the clock. When T is asserted ($T=1$), the flip flop state toggles back and forth at each active edge of the clock, and when T is de-asserted, the flip-flop keeps its current state

Characteristic Table of the T Flip Flop

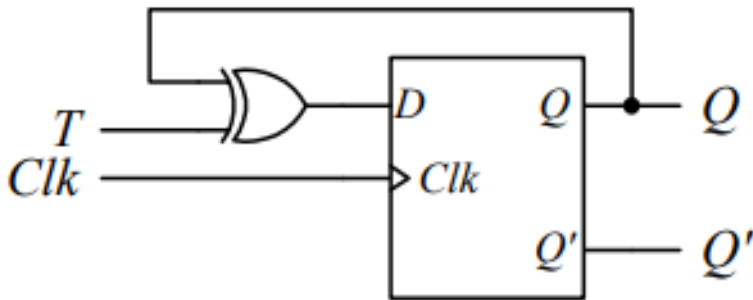
T	Q_n	Q_{n+1}	$\bar{Q}_{n+1}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

The characteristic equation of T flip flop.

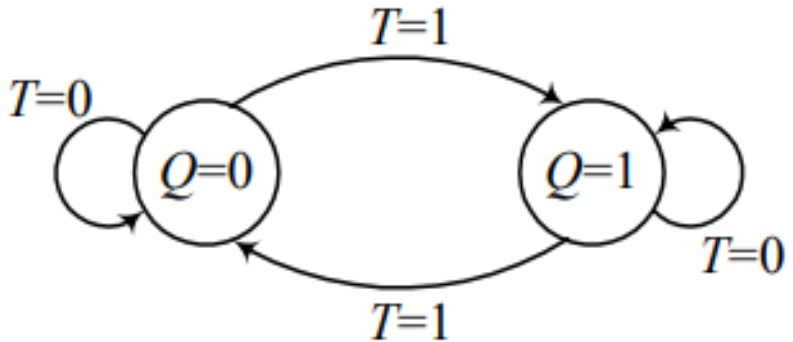
$$Q_{next} = T\bar{Q} + \bar{T}Q$$



T flip flop Logic Symbol



circuit diagram of T flip flop



State diagram of T flip flop

Excitation Table of the T Flip Flop

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

TUTORIAL QUESTIONS Question 1

- (i) Design a 3 bit synchronous up counter.
- (ii) Design a 4 bit asynchronous down counter
- (iii) Design MOD 13 ripple counter using JK flip flop.
- (iv) A 20-kHz clock signal is applied to a J-K flip-flop with $J = K = 1$. What is the frequency of the FF output waveform?
How many FFs are required for a counter that will count 0 to 25510?
What is the frequency of the output of the eighth FF when the input clock frequency is 512 kHz?
If this counter starts at 00000000, what will be its state after 520 pulses

Introduction to Digital Computer Systems II

Akingbade, Michael Adeniyi

Obafemi Awolowo University, Ile - Ife Nigeria

akingbadem@oauife.edu.ng

June 6, 2024

Overview

1 Registers

2 Counters

3 PAL and PAL

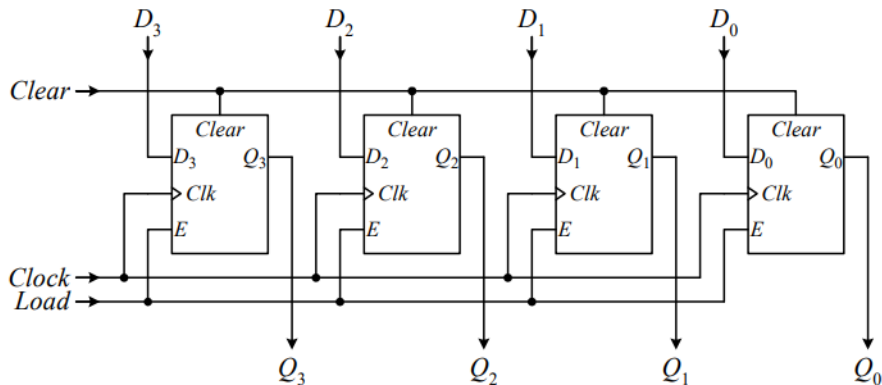
Registers

- Group of flip flop is called a register.
- When one flip flop is connected to another to increase the storage capacity. It form a register.
- The collection of Flip flop used to remember the complete history of past inputs is reffered to as the state memory.

Registers

- When we want to store a byte of data, we need to combine eight flip flops together and have them work together as a unit.
- A register is a circuit with two or more D flip flops connected together in such a way that they all worked exactly the same way and synchronized by same clock and enable signals.
- Each flip flop in the group is used to store a different bit of data.

4 - bit Register.



Circuit diagram of a 4-bit register with parallel load asynchronous clear

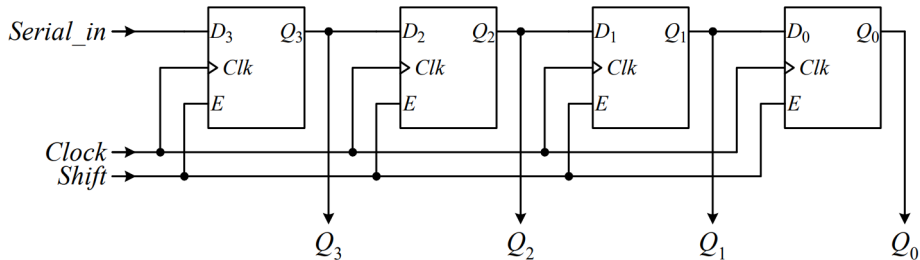
SHIFT REGISTERS

SHIFT REGISTERS

The main usage of shift register is for converting from a serial data input stream to parallel data output or vice versa .

In the first clock cycle, the first bit from the serial input stream, serial-in , gets loaded into Q3, while the original bit in Q3 is loaded in Q2, and Q2 is loaded into Q1 and so on.

Shift Register.



Circuit diagram of a 4-bit serial to parallel shift register circuit

Classification of Register

Mode of Operation

- SISO => Serial input serial output
- SIPO => Serial input parallel output
- PISO => Parallel input serial output
- PIPO => Parallel input Parallel output

Counters are for counting a sequence of values.

- Different kinds of Counters
- Up Counter
- Down Counter
- Modulo -n Counter

Counters

Counter.

- Modulo -n Counter $\Rightarrow$ Counts from decimal 0 to n-1 for example modulo -5 counter. Counter sequence in decimal 0, 1, 2, 3 and 4
- Binary Coded (BCD) Counter. $\Rightarrow$ It is a modulo -10 counter
- n - bit binary counter $\Rightarrow$ It range is 0 to $2^n - 1$ and back to 0. Where n is the number of bits used in the counter.
- Gray code counter $\Rightarrow$ The sequence is coded so that any two consequence value must differ in one bit. eg. 000, 001, 011, 010, 110, 111, 101, 100
- Ring counter $\Rightarrow$ The sequence starts with a string of 0 bits followed by one 1 bit as in 0001. This counter simply rotates the bits to the left on each count.
for example a 4 - bit ring counter sequence is 0001, 0010, 0100, 1000 and back to 0001.

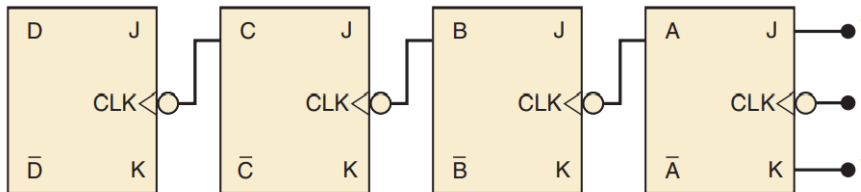
It is a counter which counts upto particular value.

Basically there are two types of counters

- Types of Counters
- Asynchronous Counter or Ripple Counter
- Synchronous Counter

Asynchronous Counter or Ripple Counter

In Asynchronous Counter or Ripple Counter the output are directly to the clock input of the next flip flop. In Asynchronous Counter the output of the first flop is connected to the clock input of the next flip flop.



Asynchronous Counter is divided into three types

- Up Asynchronous Counter
- Down Asynchronous Counter
- Up - Down Asynchronous Counter

Synchronous Counter

In Synchronous Counter clock is applied synchronous to all the flip flop. And we can get the output at any stage.

Synchronous Counter is divided into three types

- Up Synchronous Counter
- Down Synchronous Counter
- Up - Down Synchronous Counter

Step of designing 2 - bit Synchronous Counter

- Step 1 => State the number of flip flop
- Step 2 => Draw excitation table for the given flip flop.
- Step 3 => Draw the state diagram
- Step 4 => Draw the truth table.
- Step 5 => Draw the K - map
- Step 6 => Draw the logic diagram

Number of flip flop

Step 1 $\Rightarrow$ 2 flip flop for 2 bits

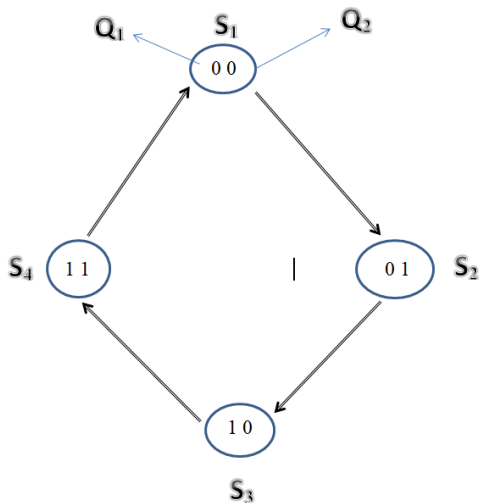
Excitation Table of the the given Flip Flop

Step 2 $\Rightarrow$ Excitation Table

Q_n	Q_{n+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

Step 3 => State Diagram

2 flip flop for 2 bits = $2^2 = 4$ number of states



Truth Table

Step 4 $\Rightarrow$ Truth table

$Q_1 Q_2$	$Q_1^* Q_2^*$	$J_1 K_1$	$J_1 K_2$
0 0	0 1	0 x	1 x
0 0	1 0	1 x	x 1
1 0	1 1	x 0	1 x
1 1	0 0	x 1	x 1

K - MAP

K - Map

For J_1

J_1

Q_1	0	1
Q_2		
0	0	x
1	1	x

$$J_1 = Q_2$$

K - MAP

K - Map

For K_1

K_1

Q_1	0	1
Q_2		
0	X	0
1	X	1

$$K_1 = Q_2$$

K - MAP

K - Map

For J_2

J_2

Q_1

Q_2

	0	1
0	1	1
1	X	X

$$J_2 = 1$$

K - MAP

K - Map

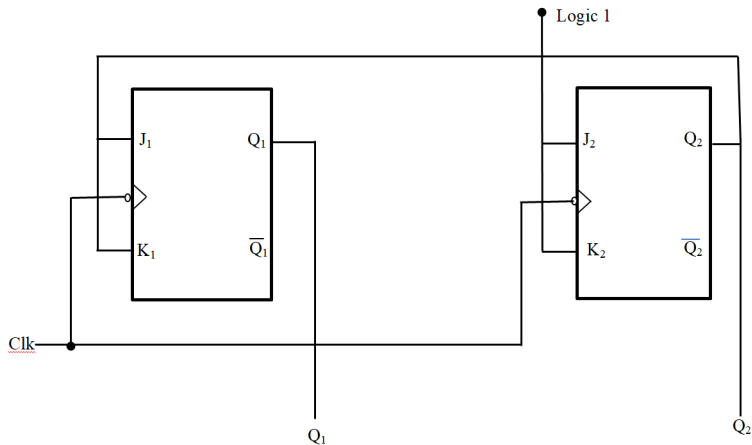
For K_2

K_2

Q_1	0	1
Q_2		
0	X	X
1	1	1

$$K_2 = 1$$

2 - bit Synchronous Up Counter



2 - bit Synchronous Up Counter

TUTORIAL QUESTIONS Question 1

- (i) Design a 3 bit synchronous up counter.
- (ii) Design a 4 bit asynchronous down counter
- (iii) Design MOD 13 ripple counter using JK flip flop.
- (iv) A 20-kHz clock signal is applied to a J-K flip-flop with $J = K = 1$. What is the frequency of the FF output waveform?
How many FFs are required for a counter that will count 0 to 25510?
What is the frequency of the output of the eighth FF when the input clock frequency is 512 kHz?
If this counter starts at 00000000, what will be its state after 520 pulses

Programmable logic Device

- PROM (Programmable Read Only Memory)
- PLA (Programmable Logic Array)
- PAL (Programmable Array Logic)

Comparison for PLA and PAL

- | • PLA | PAL |
|---------------------------|-------------------|
| • Speed reduces | More speed |
| • More flexibility | Less flexibility |
| • Less performance | More performance |
| • Hard to implement | Easy to implement |

Comparison of Programmable ROM, PLA and PAL

Programmable ROM	PLA	PAL
AND array is fixed and OR array is programmable	Both AND array & OR array are programmable	OR array is fixed & AND array is programmable
Cheaper and simple to use	Costliest and complex than PAL and ROM	Cheaper and simpler
All minterms are decoded	AND array can be programmed to desired minterms	AND array can be programmed to get desired minterms
Only boolean functions in standard sop term can be implemented using PROM	Any boolean function in SOP form can be implemented using PLA	Any boolean function in SOP form can be implemented using PAL

ASIC

Application Specific Integrated Circuit

An integrated circuit is designed to implement a specific desired application.

Programmable Logic Devices

- SPLDs (Simple Programmable Logic Devices)
- CPLDs (Complex Programmable Logic Devices)
- FPGA (Field Programmable Gate Arrays)

CPLD and FPGA are high capacity PLD.

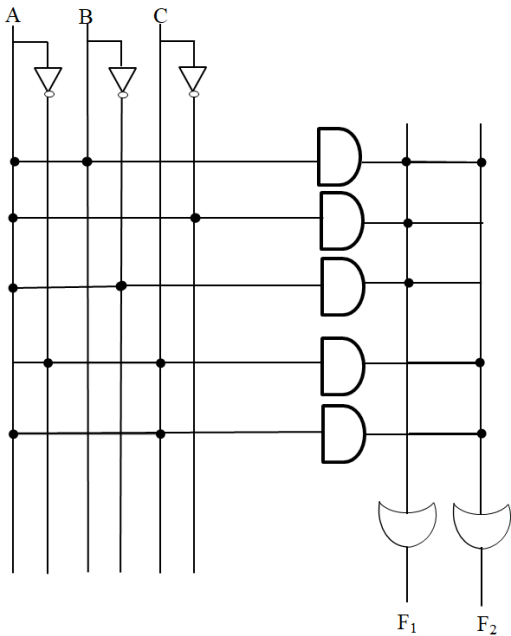
HCPLD (between thousands - Hundreds of thousands gates)

SPLD (less or equal 600 gates)

Design PLA to implement

① $F_1 = AB + A \bar{C} + A \bar{B}$

② $F_2 = AB + \bar{A}C + AC$



Designing Programmable Logic Array

- 1 Design PLA to implement

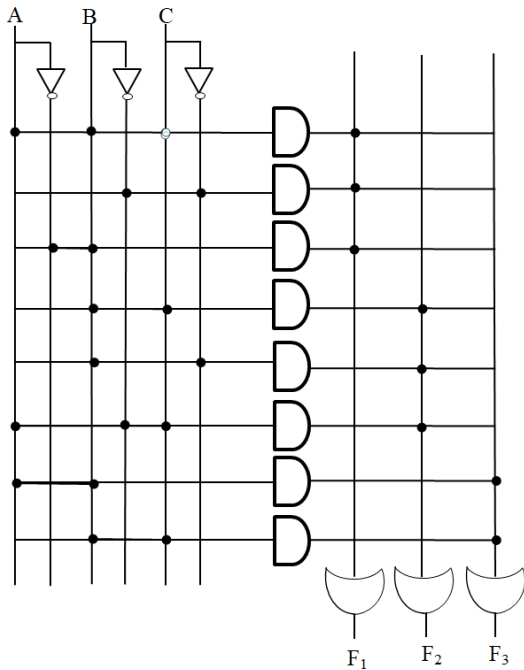
$$F_1 = \sum_m (3, 5, 6, 7) \text{ and } F_2 = \sum_m (0, 1, 2, 3, 4)$$

Design PLA to implement

① $F_1 = ABC + \bar{B} \bar{C} + \bar{A}B$

② $F_2 = BC + B\bar{C} + A\bar{B}C$

③ $F_3 = AB + BC$



Designing Programmable Logic Array

- 1 Design PLA to implement

$$F_1 = \sum_m (3, 5, 6, 7) \text{ and } F_2 = \sum_m (0, 1, 2, 3, 4)$$

K -Map fo the design

F_1

YZ	00	01	11	10
X				
0	0	0	1	0
1	0	1	1	1

$$F_1 = XY + YZ + XZ$$

$$\bar{F}_1 = \bar{X} \bar{Y} + \bar{Y} \bar{Z} + \bar{X} \bar{Z}$$

K -Map fo the design

F_2

YZ	00	01	11	10
X				
0	1	1	1	1
1	1	0	0	0

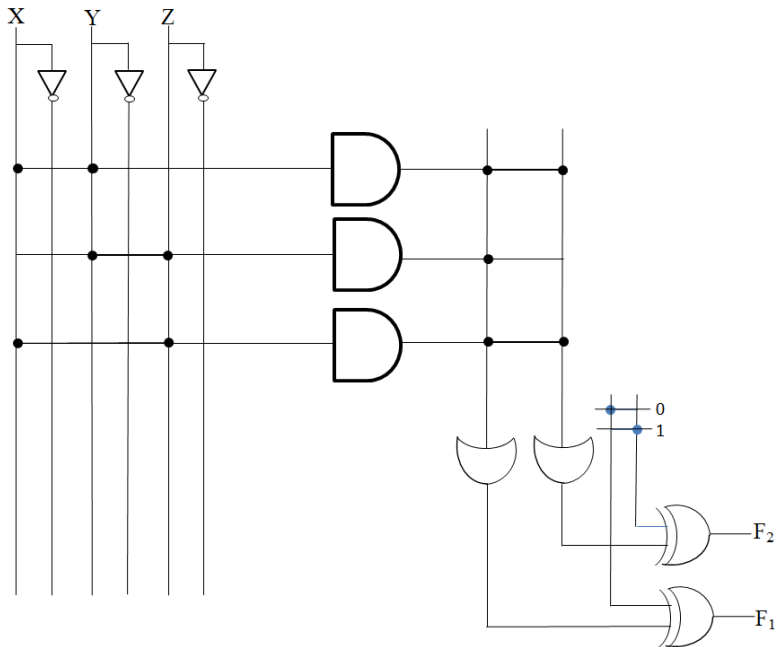
$$F_2 = \bar{X} + \bar{Y} \bar{Z}$$

$$\bar{F}_2 = XZ + XY$$

Programming Table

Product	Inputs	Output	Output
Terms	X Y Z	F_1	F_2
X Y	1 1 0	1	1
Y Z	0 1 1	1	0
X Z	1 0 1	1	1

The Circuit design that implement the PAL is on the next page



Programmable Array Logic (PAL)

Design PAL to implement

① $F_1 = ABC + \bar{A} \bar{B} \bar{C}$

② $F_2 = AB + BCD$

③ $F_3 = \bar{A} B + CD + BD$

④ $F_4 = \bar{A} \bar{B} + \bar{C} \bar{D} + \bar{B} \bar{D}$

